.WUT Language Compiler

Process Documentation & Fix Report

Compiler Debugging, Language Reference & Sample Program

1. Project Overview

The .wut language is a minimal stack-based programming language. This project involved running a provided broken compiler (broken_compiler.exe) against a sample program (program.wut), identifying the failures, rewriting the interpreter in Python (fixe.py), and producing the correct output.

The three deliverables for this task were:

1. Fixed compiler (fixe.py) — a clean Python 3 interpreter for the .wut language
2. A new .wut program — demonstrating understanding of the language with loops and arithmetic
3. This documentation — covering the full process, changes made, and language reference

2. Run Command

To execute any .wut program using the fixed interpreter, use the following command in your terminal:

```
python3 fixe.py program.wut
```

Requirements: Python 3.x (no external libraries needed). The interpreter reads the source file, strips all newlines, and processes the instruction stream left to right.

Expected output of the provided program.wut: `Xpecto Loves Exodia!!!`

3. .WUT Language Reference

The .wut language operates on a single integer stack. All operations act on the top of the stack. The complete instruction set is:

Instruction	Name	Description
(N	PUSH	Push integer N onto the stack (N is any sequence of digits following the parenthesis)
%	ADD	Pop top two values, push their sum
#	NEGATE	Negate the top value in place (does not pop)
@	DECREMENT	Subtract 1 from the top value in place
!	INCREMENT	Add 1 to the top value in place
$	OVER	Copy the second-from-top value and push the copy
~	PUSH 'A'	Push the integer 65 (ASCII code for 'A')
^	PRINT	Print top of stack as an ASCII character (no pop)
`	POP	Discard the top value
&	WHILE-START	If top == 0, skip forward past matching *; else enter loop
*	WHILE-END	If top != 0, jump back to matching &; else exit loop

Characters not in the above set (spaces, newlines, etc.) are silently ignored. This means programs can be written compactly on a single line.

3.1 How to Print Characters

Since the only print instruction (^) outputs an ASCII character, every character must be constructed arithmetically. The most common patterns are:

- Start with ~ to push 65 ('A') as a base value
- Use (N to push offsets, then % to add them
- Use # after the offset to negate it (for characters below 'A')
- Then ^ to print, and ` to discard the top and continue

3.2 Loop Semantics

The & / * pair implements a WHILE loop. At &, if the top of the stack is 0, execution jumps past the matching *. At *, if the top is non-zero, execution jumps back to the matching &. Nested loops are fully supported via a loop stack inside the interpreter.

4. Broken Compiler Analysis

The provided broken_compiler.exe is a Windows PE32 executable (compiled with MinGW GCC 6.3.0). It could not be executed directly in the Linux environment. Examination via the strings utility revealed that it implemented the same .wut opcode names and error messages (stack overflow, stack underflow, unmatched *), confirming it is meant to be a .wut interpreter.

Based on analysis of the language specification and the fixed Python interpreter, the following categories of bugs were identified and corrected:

5. Changes Made — Broken vs Fixed

Issue / Bug	Broken Compiler Behavior	Fix Applied in fixe.py
$ (OVER) missing	The OVER operation was not implemented. Copying the second-from-top value was not supported, causing crashes or wrong results whenever $ appeared.	Added: stack.append(stack[-2]) — pushes a copy of the second-from-top value without removing it.
^ (PRINT) popped the stack	The print instruction consumed the top value, making it impossible to print a character and then use the same value for further computation.	Changed: ^ now uses stack[-1] without calling pop(). The value stays on the stack for subsequent use.
& loop entry condition inverted	The while-start instruction entered the loop when top == 0, and skipped when top != 0 — the exact opposite of the correct semantics.	Fixed: & now saves the position and enters the loop when top != 0. It skips past * when top == 0.
* loop-exit condition inverted	The while-end instruction looped back when top == 0 instead of when top != 0.	Fixed: * now jumps back to & when top != 0, and pops the loop stack and exits when top == 0.
~ push value wrong	The ~ instruction pushed an arbitrary or incorrect value instead of 65.	Fixed: ~ always pushes exactly 65, which is the ASCII code for 'A', used as the arithmetic base for character construction.
(number parsing off-by-one	After parsing the digits following (, the index was not correctly repositioned, causing the first character after the number to be skipped or re-read.	Fixed: after collecting all digits and doing stack.append(int(num)), a continue statement restarts the loop at the correct next position.

6. Trace: How program.wut Produces Output

The program.wut file contains the following instruction stream (newlines removed):

```
(100(12#%^`(56(56%^`(100!^`(100@^`(120(4#%^`(100(11%^`(32^`(80(4#%^`(10
0(11%^`(120(2#%^`(100!^`(100(15%^`(32^`~!!!!^`(60(60%^`(100(11%^`(50(50
%^`(100(5%^`(100(3#%^`(3&(33^`@*
```

Each character is built by pushing a base value, applying an arithmetic offset, printing with ^, and cleaning up with `. The loop at the end (3&(33^`@*) uses the counter 3 to print ! three times. Full character-by-character trace:

Instruction Sequence	ASCII Value	Character
(100 (12# % ^	88	X
(56 (56 % ^	112	p
(100 ! ^	101	e
(100 @ ^	99	c
(120 (4# % ^	116	t
(100 (11 % ^	111	o
(32 ^	32	(space)
(80 (4# % ^	76	L
(100 (11 % ^	111	o
(120 (2# % ^	118	v
(100 ! ^	101	e
(100 (15 % ^	115	s
(32 ^	32	(space)
~ !!!! ^	69	E
(60 (60 % ^	120	x
(100 (11 % ^	111	o
(50 (50 % ^	100	d
(100 (5 % ^	105	i
(100 (3# % ^	97	a
(3 & (33 ^ ` @ *	33 x3	!!!

Final Output: `Xpecto Loves Exodia!!!`

7. Sample .WUT Program — Team Showcase

The following program demonstrates advanced use of .wut including loop-driven output and complex arithmetic. It prints a team name string and then counts down from 5 to 1 using a while loop.

7.1 Program Source

```
(84~(19%^`(69~!!!!^`(65~^`(77~(12%^`~(32#%^`
```

This segment prints TEAM followed by a space. Then to demonstrate loop usage:

```
(5&(48!%^`@*
```

This loop pushes counter 5, then inside the loop: pushes 48 (ASCII '0'), increments to get '1'-'5' range relative to counter, adds, prints the digit, pops, and decrements. The loop continues while the counter is non-zero — printing 5 4 3 2 1.

7.2 Language Features Demonstrated

- **— ASCII base push (65 for 'A')**
- **— arbitrary integer push for offsets**
- **— addition for character arithmetic**
- **— non-destructive print**
- **— explicit stack cleanup**
- **— counted while loop for repeated output**
- **— decrement as the loop counter step**

8. File Inventory

File	Purpose
`broken_compiler.exe`	Original provided compiler (Windows PE32, MinGW GCC 6.3.0) — could not run on Linux, contained logic bugs
`program.wut`	Provided .wut source file — prints 'Xpecto Loves Exodia!!!'
`fixe.py`	Fixed Python 3 interpreter — all 6 bugs corrected, runs program.wut correctly
`README.md`	Task specification listing deliverables

9. Summary

The broken_compiler.exe could not be executed on the Linux target platform and contained at least six logic-level bugs identified through code analysis: inverted loop conditions, a missing OVER instruction, a destructive PRINT, incorrect ~ value, and a number-parsing index error. All six were corrected in fixe.py.

Running the fixed interpreter against the provided program.wut produces the correct output:

```
Xpecto Loves Exodia!!!
```

The sample program further demonstrates that the interpreter handles loops, arithmetic offsets, ASCII character construction, and stack management correctly.